

Deep Study Guide

The Logic Advocate Tutor

From philosophical roots to every Python decision. Built for minute-detail mastery.

Adel Ashraf Mohamed Kamel Emam - GUC, SS26

Sixteen parts, ~80 dense pages. Glossary, mock viva, 100+ practice questions. Re-read in topic chunks.

Table of Contents

Part 1.	Big-Picture Orientation
Part 2.	Pre-History: Why Argumentation Theory Exists
Part 3.	Dung (1995) in depth
Part 4.	The Extensions
Part 5.	Intelligent Tutoring Systems Background
Part 6.	LLM Debate Research and Ku (2025)
Part 7.	System Architecture in depth
Part 8.	The Engine, line by line
Part 9.	The MDP, line by line
Part 10.	The AI Layer in depth
Part 11.	Every View and Feature
Part 12.	Evaluation, every number explained
Part 13.	Threats to Validity and Future Work
Part 14.	Glossary
Part 15.	Mock Viva Script
Part 16.	Question Bank by Topic

Part 1 - Big-Picture Orientation

Before any formulas, fix the project in your head as one sentence: the Logic Advocate Tutor is a Streamlit application in which a learner debates either another human or an LLM, with every move judged by a symbolic bipolar gradual argumentation engine and every hint chosen by a small Markov Decision Process. The combination of symbolic verdict and neural prose is what the thesis claims is novel and pedagogically defensible.

Why the project exists

Conversational AI is excellent at generating plausible argumentative prose but unreliable at admitting defeat. Ku et al. (2025) measured this and called it rhetorical rigidity. A student debating a chatbot cannot trust the chatbot's claim that 'you have a good point' because the chatbot is not running a verifier - it is producing fluent text whose self-evaluations are themselves fluent text. So the central engineering bet of the project is: if you cannot make the LLM honest, take honesty away from it. Move the verdict computation into deterministic Python that the LLM cannot see, write to, or influence. The UI then reads the engine and never the LLM. The LLM literally cannot lie because the LLM is not asked.

Why the project is academically interesting

Two reasons. First, this is one of very few deployed argumentation tutors anywhere - argumentation theory is 30 years old with very few real systems built on top of it. Second, the project sits across two research traditions (symbolic argumentation and intelligent tutoring) and uses a third (LLMs) as the worker layer, with an explicit guarantee about which layer is in charge of which kind of decision.

The symbolic-neural split is the central design choice

The symbolic side knows about graphs, scores, statuses, policies. The neural side knows about prose and persuasion. The system uses the neural side for what it is good at (writing prose, scoring persuasiveness, identifying implicit premises) and the symbolic side for what neural systems are demonstrably bad at (consistent verdicts, multi-step reasoning over structured graphs, reproducible behaviour). The boundary is enforced in code: the prompt that goes to the LLM contains the transcript but not the engine's scores; the UI widget that shows the verdict reads the engine and not the LLM's claim.

One-sentence pitch: 'I built a Streamlit debate tutor in which the AI writes the prose but a deterministic bipolar gradual argumentation engine decides who is winning, so the LLM is literally unable to lie about the verdict.'

Part 2 - Pre-History: Why Argumentation Theory Exists

2.1 Classical logic and its limits

Classical propositional and first-order logic has a powerful feature: monotonicity. If premises P entail conclusion C , then adding more premises cannot retract C . Once derived, always derivable.

Monotonicity: if P entails C , then $P \cup \{Q\}$ entails C for any Q .

This is wonderful for math, terrible for everyday reasoning. 'Tweety is a bird, therefore Tweety flies.' Adding 'Tweety is a penguin' should retract the conclusion. Classical logic does not let you.

Why this matters for argumentation

Argumentation is fundamentally non-monotonic. A new argument can defeat a previously accepted one. So argumentation needs a framework where conclusions retract as new information arrives.

2.2 Toulmin's model (1958)

Stephen Toulmin published 'The Uses of Argument' in 1958. He proposed six argument components: CLAIM (what to establish), DATA (facts), WARRANT (inference rule), BACKING (support for the warrant), QUALIFIER (modal hedge), REBUTTAL (exception condition). The novelty was the explicit modelling of WARRANT and REBUTTAL - the seed of defeasibility.

Why your project does not use Toulmin directly

Mapping free-text arguments to all six components is itself an unsolved NLP problem. Dung's abstraction goes the opposite direction: forget internal structure, model only the attack relation between arguments. Less expressive but tractable - that trade-off made Dung the dominant computational framework.

2.3 The 1980s non-monotonic crisis

Reiter's Default Logic (1980), McCarthy's Circumscription (1980), Moore's Autoepistemic Logic (1985) all attacked non-monotonic reasoning. Powerful but heavy and hard to give intuitive semantics. By the early 1990s the field needed a unifying perspective.

2.4 The bridge to Dung

Phan Minh Dung's 1995 paper made a startling claim: most popular non-monotonic logics can be re-cast in a single, much simpler framework where arguments are abstract nodes and only the attack relation matters. This is the framework your project sits on.

If asked 'why argumentation theory at all?' - it exists because classical logic is monotonic and cannot represent defeasible reasoning. Toulmin gave vocabulary; Dung gave a computational framework. Everything I built sits on that lineage.

Part 3 - Dung (1995) in depth

3.1 Motivation and abstraction

Dung's central observation: arguments and their attack relationships can be modelled as a directed graph without losing the essential reasoning. His framework subsumes Reiter's Default Logic, Pollock's defeasible inheritance, n-person games, and logic programming's stable-model semantics. That breadth is why the paper is foundational.

3.2 Five definitions to memorise

D1 Argumentation Framework: $F = \langle A, R \rangle$ where A is a finite set of arguments and $R \subseteq A \times A$ is the attack relation.
D2 Conflict-free set: $S \subseteq A$ is conflict-free if no a, b in S has a attacks b .
D3 Acceptable argument: a is acceptable w.r.t. S if for every attacker b of a there exists c in S with c attacks b .
D4 Admissible set: conflict-free S in which every element is acceptable w.r.t. S .
D5 Extensions: Complete (admissible, contains every defended argument), Preferred (maximal complete), Stable (admissible and attacks every non-member), Grounded (minimal complete).

3.3 Semantics zoo

Grounded is the most sceptical, always unique - the safe default. Preferred is more daring; multiple preferred extensions can coexist, each a coherent world-view. Stable partitions A into IN and OUT completely; may not exist. Complete is the umbrella.

Your engine's 0.5 threshold approximates the grounded extension in the classical limit. So grounded is the Dung-side counterpart your engine generalises.

3.4 Grounded extension worked example

$A = \{a1, a2, a3\}$
 $R = \{ (a2, a1), (a3, a2) \}$

$a3 \rightarrow a2 \rightarrow a1$

Round 1: $a3$ has no attackers, so $a3$ in G .
Round 2: $a2$ is attacked by $a3$ (in G), so $a2$ is OUT.
 $a1$'s only attacker ($a2$) is OUT, so $a1$ is IN.
Round 3: no changes. $G = \{a3, a1\}$.
Three-valued: $a3=IN$, $a2=OUT$, $a1=IN$.

3.5 Why Dung still dominates 30 years later

Three reasons. The framework is irreducible: A and R are the minimum. The semantics handle multiple defensible positions. Every subsequent extension is presented as 'classical Dung plus one ingredient,' which makes the literature navigable.

Part 4 - The Extensions

4.1 Besnard & Hunter (2001) - h-categorizer

They wanted a numerical evaluator for Dung frameworks. Every argument gets a score in $(0, 1]$ as the unique fixed point of:

$$h(a) = 1 / (1 + \sum_{b \text{ atk } a} h(b))$$

Isolated argument: $h = 1$. One attacker at full strength: $h = 1/2$. Two attackers: $h = 1/3$.

Existence and uniqueness of the fixed point

The map $T(h)(a) = 1/(1 + \sum h(b))$ is a contraction in L_∞ under mild graph assumptions. By Banach, T has a unique fixed point and the iteration $h_{t+1} = T(h_t)$ converges from any starting vector.

Relation to your engine

Empty support, $w=1$, $\mu=1$: your formula collapses to $s(a) = 1/(1+\alpha) = h\text{-categorizer}$. So Besnard-Hunter is your classical limit.

4.2 Amgoud & Cayrol (2002) - Preference-Based AF

$$F = \langle A, R_{\text{att}}, \succ \rangle$$

a defeats b iff a attacks b AND not $(b \succ a)$.

Stronger arguments resist defeat from weaker ones. You replace the partial order with integer weights in $\{1, \dots, 25\}$. Softened from strict drop to a ratio: weighted attack divided by target's weight. Pedagogically preferable - weak attacks land partially, not silently.

Why integer 1-25 specifically

The LLM rates persuasiveness on 1-25 in the pipe protocol; same scale removes a translation step. Integers display cleanly. 25 is a soft cap that prevents a single rhetorically dominant move from making every counter mathematically useless.

4.3 Bench-Capon (2003) - Value-Based AF

$$F_{\text{VAF}} = \langle A, R_{\text{att}}, V, \text{val}, \succeq_{\text{aud}} \rangle$$

a defeats b for aud iff $a \text{ atk } b$ AND not $(\text{val}(b) \succeq_{\text{aud}} \text{val}(a))$.

Bench-Capon's insight: the same framework produces different verdicts for different audiences. Real debate outcomes ARE audience-relative.

Your operationalisation: four values, four multipliers.

$$\mu(\text{Fact})=1.2, \mu(\text{Logic})=1.1, \mu(\text{Ethics})=1.0, \mu(\text{Emotion})=0.8$$

These encode a fact-leaning rational audience (the academic default). The ratio $\mu(v(b))/\mu(v(a))$ scales every attack and support. A Fact attack on Emotion multiplies by $1.2/0.8 = 1.5$; the reverse 0.667.

Why these specific numbers?

Hand-tuned. The ordering matches Bench-Capon's rationalist audience. 1.5x for the strongest mismatch is enough to flip borderline verdicts without dominating. Future work: learn per-classroom.

4.4 Cayrol & Lagasquie-Schiex (2005) - Bipolar AF

$$F_{\text{BAF}} = \langle A, R_{\text{att}}, R_{\text{sup}} \rangle$$

(b, a) in R_{sup} means ' b reinforces a '.

Without supports, your only way to strengthen a claim is to attack its attackers (indirect defence). With supports, you also reinforce directly. Pedagogically different moves. Ablation: 26.7% of verdicts flip when

supports are off.

4.5 Gradual semantics - Cayrol (2005), Amgoud (2017)

Real-valued strengths replace categorical IN/OUT.

$s_{t+1}(a) = \phi(w(a), \{s_t(b) : b \text{ atk } a\}, \{s_t(c) : c \text{ sup } a\})$

- h-categorizer: $s = 1/(1 + \sum_{\text{atk}} s)$
- Card-based: $s = w(a)/(1 + \text{count}_{\text{atk}}(a))$
- Yours: $s = (1 + \sigma/w)/(1 + \alpha/w)$, capped at 1

Convergence

For ϕ continuous and monotone, on the compact $[0,1]^{|A|}$, iteration converges to a fixed point under mild graph assumptions. Empirically: 2-3 iterations on all 60 scenarios.

Why gradual matters for a tutor

Telling a student 'OUT' is uninformative; they cannot distinguish narrowly defeated from thoroughly demolished. Continuous score gives gradient feedback. Ablation: 41.7% verdict change between gradual and classical-grounded.

Part 5 - Intelligent Tutoring Systems Background

5.1 Brief history of ITS

ITS adapts instruction to individual students. SCHOLAR (Carbonell 1970), cognitive tutors based on ACT-R in the 1980s, Bayesian Knowledge Tracing (Corbett & Anderson 1995) as the probabilistic standard.

Why ITS matters to your project

Your hint MDP is an ITS technique applied to debate. State tracks the strategic situation, actions are hint strategies, reward ties to engine verdict.

5.2 Bayesian Knowledge Tracing (brief)

BKT models skill mastery as a hidden binary variable. Four params: $P(L_0)$ prior, $P(T)$ unmastered-to-mastered, $P(G)$ guess, $P(S)$ slip.

Why you did not use BKT

BKT presumes a target skill to teach. Your tutor does not teach a fixed skill; it provides a deliberation environment. The student's state is the local debate situation, not 'probability of mastering debate skill.' An MDP over engine-derived state fits more naturally.

5.3 Stamper (2011) Hint Factory deep dive

Stamper, Eagle, Barnes, Croy built the Hint Factory for the Deep Thought logic tutor. Mined historical student solution attempts into a graph: nodes are problem states, edges are student moves with frequency and outcome labels. New student stuck $\rightarrow$ look up state $\rightarrow$ recommend the highest-historical-success next move.

Design principles you adopted

- **On-demand:** hints appear only on request.
- **Non-evaluative:** hint suggests direction, not 'you're wrong.'
- **Adaptive:** recommendation reflects prior students' success, not a hand-coded heuristic.

Empirical results from Stamper et al.

Students with Hint Factory access completed significantly more problems and were less likely to abandon. Hint utility above zero. Hint frequency dropped as competence grew - no over-reliance.

How yours differs from theirs

- Their P is empirical; yours is expert-informed (no logged data yet, but stub for empirical re-estimation exists).
- Their state is problem-position; yours is debate-situational.
- Their action space is concrete next-moves; yours is hint STRATEGIES (the LLM writes the prose). You sit one level above the prompt-LLM.

Part 6 - LLM Debate Research and Ku (2025)

Their experimental setup

Ku and colleagues assigned LLMs (GPT-4, Claude, LLaMA-3) debate stances on contested propositions and observed multi-turn dialogues. Annotated for rhetorical rigidity (refusing to concede) and implicit premise blindness (failing to acknowledge unstated assumptions).

Their main findings

- **Rhetorical rigidity is universal.** Across LLMs, once assigned a stance, models defend flawed reasoning rather than concede. Concession rate below 5%.
- **Implicit premises are routinely missed.** Even when prompted to check, models rarely surface the unstated assumption.
- **Adversarial setup amplifies both effects.** Two LLMs on opposite stances produce more rigid and more implicit-blind arguments than single-LLM analytic mode.

What your project adds on top of Ku

- **You take the verdict out of the LLM's hands entirely.** Their study assumes the LLM judges. Your system never asks. The engine judges.
- **You expose implicit premises through a UI feature.** Check Assumptions surfaces the unstated premise with a confidence score, and (if accepted) attaches it as an explicit supporting node.

Worth saying aloud: Ku diagnoses. They propose no treatment. Check Assumptions is, as far as I am aware, the only deployed argumentation tutor that turns the Ku diagnosis into a UI feature that surfaces the missed premise into the formal graph.

Adjacent literature worth knowing

- **Du et al. (2024)** - Multi-agent debate for factuality.
- **Khan et al. (2024)** - Persuasive LLMs supervise truth better.
- **Irving, Christiano & Amodei (2018)** - AI safety via debate (earlier conceptual proposal).

Part 7 - System Architecture in depth

Why four layers

Four is the minimum number that separates presentation, dialogue rules, symbolic computation, and neural inference. Fewer conflates; more adds overhead.

Layer 1 - Presentation (Hybrid)

Files: `views/local_debate.py` (~1019), `views/ai_vs_ai.py` (~352), `views/engine_inspector.py` (~393), `views/_styles.py` (~294), `app.py` (~115).

Renders chat bubbles, momentum bar, Graphviz logic map, engine inspector, composer. Reactive: re-renders on every interaction.

Layer 2 - Dialogue (Hybrid)

Files: parts of `app.py`, `_process_move()` in `local_debate.py`, the per-mode state stash/restore.

Responsibilities: enforce turn order, validate composer, run the 60-second blitz timer, route between three game modes, preserve per-mode state.

Layer 3 - Logic (Symbolic)

Files: `logic_engine.py` (~218), `hint_mdp.py` (~328).

Maintains the bipolar gradual framework, runs fixed-point relaxation, derives IN/OUT, exposes MDP state, recommends hint strategy. Pure deterministic Python. No I/O. No LLM. No Streamlit.

Unit-testable in isolation. 101 passing assertions. Reproducible: same input -> same output, every time.

Layer 4 - AI (Neural)

File: `ai_agent.py` (~327).

Wraps the Groq endpoint. Transcribes audio with Whisper. Generates counter-arguments with LLaMA-3 (text or vision). Generates hints by rendering an MDP-chosen prompt. Detects implicit premises.

Input: the transcript. Output: text and integer scores. The input never includes the engine's scores. This is the boundary that enforces the pedagogical guarantee.

One move's flow through the four layers

- **1. Presentation:** user types, picks tag and target, clicks arrow. Streamlit captures the submission.
- **2. Dialogue:** `_process_move` validates, calls `engine.add_argument`, `engine.add_direct_attack/add_support`, then `evaluate_semantics`.
- **3. Logic:** engine runs ten rounds of relaxation, computes scores, thresholds to statuses.
- **4. AI:** if HvAI and the move was the human's, `generate_counter_argument` is called with (transcript, latest_text). LLM returns pipe line. Parser extracts (LLM_score, LLM_text, human_score, value_tag).
- **5. Dialogue (again):** AI reply added as another engine node. Engine re-runs.
- **6. Presentation:** Streamlit re-renders. Every widget reads `engine.scores` and `engine.statuses`.

The boundary lives in `_build_transcript()`. That function reads messages and never engine state. Adding engine state to the prompt would break the guarantee.

Part 8 - The Engine, line by line

8.1 Full formula derivation

Build up from h-categorizer to your final formula in four stages.

Stage 0 - h-categorizer

$$s(a) = 1 / (1 + \sum_{b \text{ atk } a} s(b))$$

Stage 1 - add per-argument weights

$$s(a) = 1 / (1 + \sum_{b \text{ atk } a} s(b) * w(b) / w(a))$$

Stage 2 - add value tag multipliers

$$s(a) = 1 / (1 + \sum_{b \text{ atk } a} s(b) * w(b) * \mu(v(b)) / \mu(v(a)) / w(a))$$

Stage 3 - add support relation

$$\alpha(a) = \sum_{b \text{ atk } a} s(b) * w(b) * \mu(v(b)) / \mu(v(a))$$

$$\sigma(a) = \sum_{b \text{ sup } a} s(b) * w(b) * \mu(v(b)) / \mu(v(a))$$

$$s(a) = (1 + \sigma(a) / w(a)) / (1 + \alpha(a) / w(a))$$

Stage 4 - cap at 1

$$s(a) = \min(1, (1 + \sigma(a) / w(a)) / (1 + \alpha(a) / w(a)))$$

Final formula. Bipolar, gradual, weighted, value-tagged.

Sanity check: classical limit

Empty support, $w=1$, $\mu=1$:

$$s(a) = \min(1, 1 / (1 + \alpha(a))) = \text{h-categorizer}.$$

8.2 Five worked examples

Example 1 - Isolated claim

$A=\{a1\}$, no relations. $\alpha=0$, $\sigma=0$. $s(a1)=1$. IN.

Example 2 - Single attack (classical limit)

$A=\{a1, a2\}$. $a2 \text{ atk } a1$. $w=1$, $\mu=1$.

$t=0$: $s(a1)=s(a2)=1$.

$t=1$: $\alpha(a1)=1$. $s(a1)=1/2=0.5$. $a2$ stays 1.

Fixed point. $s(a1)=0.5$ (border IN), $s(a2)=1$.

Example 3 - Two attackers (classical limit)

$A=\{a1, a2, a3\}$. $a2, a3 \text{ atk } a1$. uniform w, μ .

$t=1$: $\alpha(a1)=2$. $s(a1)=1/3=0.333$. OUT.

Matches h-categorizer: $1/(1+1+1)=1/3$.

Example 4 - Value-tag asymmetry

$A=\{a1, a2\}$. $a2 \text{ atk } a1$. $w=1$.

Case A: $v(a1)=\text{Fact}(1.2)$, $v(a2)=\text{Emotion}(0.8)$.

$$\alpha=1*1*0.8/1.2=0.667. s=1/1.667=0.60. \text{ IN.}$$

Case B (swap): $v(a1)=\text{Emo}$, $v(a2)=\text{Fact}$.

$$\alpha=1*1*1.2/0.8=1.5. s=1/2.5=0.40. \text{ OUT.}$$

Same graph, opposite verdict, purely from value asymmetry.

Example 5 - Bipolar support rescues a defeated claim

$A=\{a1, a2, a3\}$. $a2 \text{ atk } a1$, $a3 \text{ sup } a1$. $w=10$, Logic.

$\alpha(a1)=10$, $\sigma(a1)=10$.

$$s(a1) = (1+1)/(1+1) = 1.0. \text{ IN.}$$

Without a_3 : $s(a_1)=1/2=0.5$. Borderline.
This is the kind of case the 26.7% ablation counts.

8.3 Annotated evaluate_semantics()

```
def evaluate_semantics(self):
    # Init at 1.0 - relaxation only pushes scores DOWN as attacks
    # accumulate. Starting at 0 would slow convergence.
    self.scores = {mid: 1.0 for mid in self.nodes}

    for iteration in range(10):
        new_scores = {}
        # Synchronous: read from self.scores (PREVIOUS iteration),
        # never from new_scores. Makes the result independent of
        # node insertion order.
        for mid in self.nodes:
            t_val = self.nodes[mid].get("value_tag", "Logic")
            target_weight = max(1, self.nodes[mid]["weight"])

            # ATTACK CONTRIBUTION
            attack_sum = 0.0
            for atk, tgt in self.attacks:
                if tgt == mid and atk in self.nodes:
                    a_val = self.nodes[atk].get("value_tag", "Logic")
                    mult = VALUE_WEIGHTS[a_val]/VALUE_WEIGHTS[t_val]
                    attack_sum += self.scores[atk] * \
                        self.nodes[atk]["weight"] * mult

            # SUPPORT CONTRIBUTION (mirror of attacks)
            support_sum = 0.0
            for sup, tgt in self.supports:
                if tgt == mid and sup in self.nodes:
                    s_val = self.nodes[sup].get("value_tag", "Logic")
                    mult = VALUE_WEIGHTS[s_val]/VALUE_WEIGHTS[t_val]
                    support_sum += self.scores[sup] * \
                        self.nodes[sup]["weight"] * mult

            # COMPOSE
            score = (1.0 + support_sum/target_weight) / \
                (1.0 + attack_sum/target_weight)
            new_scores[mid] = min(1.0, score) # cap

            # EARLY TERMINATION
            max_delta = max(abs(new_scores[m]-self.scores[m])
                            for m in self.nodes)
            self.scores = new_scores
            if max_delta < 1e-6:
                break

    # THRESHOLD
    for mid in self.nodes:
        self.statuses[mid] = "IN" if self.scores[mid]>=0.5 else "OUT"
```

8.4 Edge cases and invariants

Empty engine

for-loops iterate over nothing. Returns gracefully. Empty dicts.

Self-attacker

Iterates to $x = 1/(1+x)$, giving $x = (\sqrt{5}-1)/2 = 0.618$.

Support cycle

Scores grow above 1 except for the cap, saturating at 1.

Attack/support asymmetry

alpha in the denominator (suppresses), sigma in the numerator (boosts). Design choice, not forced by any paper.

Invariant: for every node a at every iteration, $0 < s(a) \leq 1$.

Part 9 - The MDP, line by line

9.1 MDP refresher

A Markov Decision Process is a 5-tuple $\langle S, A, P, R, \gamma \rangle$.

- **S** = states.
- **A** = actions in each state.
- **P(s'|s,a)** = transition probability.
- **R(s,a,s')** = immediate reward.
- **gamma in [0,1]** = discount factor.

Policy π : $S \rightarrow A$. Value $V_{\pi}(s)$ = expected discounted future reward starting in s under π . Optimal policy π^* maximises V for every s .

9.2 Value iteration derivation

Bellman optimality:

$$V^*(s) = \max_a \sum_{s'} P(s'|s,a) * [R(s,a,s') + \gamma V^*(s')]$$

Iterate from $V_0 = 0$:

$$V_{t+1}(s) = \max_a \sum_{s'} P(s'|s,a) * [R(s,a,s') + \gamma V_t(s')]$$

Under finite S and $\gamma < 1$, $V_t \rightarrow V^*$ uniformly. Geometric rate with ratio γ . With $\gamma=0.9$ and threshold $1e-9$, ~197 iterations worst case. For 18 states: microseconds.

Optimal policy: $\pi^*(s) = \arg\max_a [\text{same sum}]$.

hint_mdp.py caches V^* and π^* at module import. Runtime lookup is $O(1)$ dict access.

9.3 Our specific tuple

State space - 18 states, three buckets

```
own_status:      {WINNING, TIED, LOSING}    -> 3
opponent_pressure: {LOW, HIGH}              -> 2
hint_streak:     {ZERO, ONE, TWO_PLUS}      -> 3
|S| = 18
```

Why these three buckets?

- **own_status** ties to the reward signal.
- **opponent_pressure** separates 'comfortable win' from 'precarious win.' Without it, no signal to **VALUE_REFRAME**.
- **hint_streak** implements Stamper's non-evaluative principle: spam triggers fatigue.

Action space - four hint strategies

- **STRATEGIC_ATTACK** - opponent's weakest premise.
- **DEFENSIVE_REINFORCE** - strengthen own claim.
- **VALUE_REFRAME** - switch to a value tag the opponent is weaker on.
- **STEP_BACK** - reconsider the main claim.

Transition probability - design intuitions

- **STRATEGIC_ATTACK** works best when **LOSING** (high-variance recovery).

- DEFENSIVE_REINFORCE works best when TIED (consolidate margin).
- VALUE_REFRAKE works best under HIGH pressure (escape frame).
- STEP_BACK has highest WINNING from LOSING (max information).
- Every action loses effectiveness with streak (0.85x ONE, 0.65x TWO_PLUS).

Reward - the link to the engine

```
R(s,a,s') = REWARD_BY_NEXT_STATUS[next.own_status] + HINT_COST
            = (+1 WINNING, 0 TIED, -1 LOSING) - 0.05
```

own_status comes from derive_state -> engine.statuses[Msg_1].
MDP cannot reward a hint the ENGINE does not endorse.

9.4 Annotated solver

```
DISCOUNT = 0.9
EPSILON = 1e-9
```

```
def solve():
    V = {s: 0.0 for s in STATES}
    while True:
        new_V = {}
        for s in STATES:
            best = -float("inf")
            for a in ACTIONS:
                expected = sum(
                    prob * (reward(s,a,ns) + DISCOUNT*V[ns])
                    for ns, prob in
                        _transition_distribution(s,a).items()
                )
                if expected > best:
                    best = expected
            new_V[s] = best
        if max(abs(new_V[s]-V[s]) for s in STATES) < EPSILON:
            break
        V = new_V
    policy = {}
    for s in STATES:
        best_a, best_v = None, -float("inf")
        for a in ACTIONS:
            expected = sum(
                prob * (reward(s,a,ns) + DISCOUNT*V[ns])
                for ns, prob in _transition_distribution(s,a).items()
            )
            if expected > best_v:
                best_v, best_a = expected, a
        policy[s] = best_a
    return V, policy

_V_STAR, _POLICY = solve()    # cached at import

def choose_action(state):
    return _POLICY[state]     # 0(1) lookup
```

Annotated derive_state()

```
def derive_state(engine, messages, learner_side, recent_hints):
    # 1. Own status from engine verdict on Msg_1
    main_claim_score = engine.scores.get("Msg_1", 0.5)
    if learner_side == "A":
        own = ("WINNING" if main_claim_score>=0.65 else
              "LOSING" if main_claim_score< 0.40 else "TIED")
    else:
        # Side B: polarity flipped.
        own = ("WINNING" if main_claim_score< 0.40 else
              "LOSING" if main_claim_score>=0.65 else "TIED")
```



```
# 2. Opponent pressure
last_opp = next((m for m in reversed(messages)
                 if m["side"] != learner_side), None)
if last_opp:
    opp_score = engine.scores.get(last_opp["mid"], 0.5)
    pressure = "HIGH" if opp_score >= 0.65 else "LOW"
else:
    pressure = "LOW"

# 3. Hint streak
streak = ("ZERO" if recent_hints == 0 else
          "ONE" if recent_hints == 1 else "TWO_PLUS")

return (own, pressure, streak)
```

Part 10 - The AI Layer in depth

10.1 The grounding mechanism - three explanations

Explanation 1: the LLM is not asked

The counter prompt asks for three things: a rating of the human's argument, a rating of its own counter, and the counter's text. It does NOT ask 'who is winning?' That question is never put to the LLM. The engine answers.

Explanation 2: the engine runs after every move

After every move, `_process_move` calls `evaluate_semantics`. A knockout argument immediately downgrades the AI's previous claim. The UI reads that downgraded score. No LLM call can revert it.

Explanation 3: the UI never reads LLM-supplied verdict fields

The LLM does supply self-ratings, but those go IN as weights, not OUT as verdicts. Widgets that show 'who is winning' read `engine.statuses`. Even '99|1|Logic| CRUSHED THIS' would not change the verdict label.

Memorise: the LLM cannot lie because it is not in the position of having to make the claim that the verdict-display would contradict. The guarantee is enforced by separation of concerns, not by trusting the LLM.

10.2 Prompt engineering walkthrough

Counter prompt sections

- Setup line: 'You MUST engage with ANY debate topic' (counteracts refusal behaviour on sensitive topics).
- Rules A-D: behavioural rules.
- Style: chatty, conversational, length up to LLM.
- Transcript and latest argument.
- Output protocol: exactly one pipe-formatted line.

Why each rule is what it is

Rule A (accept personal experience): the LLM was rejecting 'I drink 1 litre a day' on the grounds of no peer-reviewed citation. Pedantic-tutor failure. The rule says treat self-reports as testimony.

Rule B (score honestly): the LLM self-favours - rates itself 19 against human's 8 even when human is clearly stronger. The rule names the bias and tells the LLM to resist.

Rule C (concede when no real counter): the LLM is reluctant to lose. CONCEDE is the only honest exit. Without explicit license, the LLM produces a weak counter rather than admit defeat.

Rule D (stay focused): otherwise the LLM nitpicks typos and casual phrasing. Tells it to engage with content.

10.3 The pipe parser, regex by regex

```
_PIPE_RE = re.compile(  
    r"(\d{1,2})\s*\|(\d{1,2})\s*\|(\d{1,2})\s*\|(\d{1,2})\s*"   
    r"(Logic|Fact|Ethics|Emotion)\s*\|(\s*([^\n]+))",   
    re.IGNORECASE)
```

- `(\d{1,2})` - human score, 1-2 digits.

- `\s*\|s*` - pipe with optional whitespace.
- `(\d{1,2})` - AI score, 1-2 digits.
- `(Logic|Fact|Ethics|Emotion)` - case-insensitive tag.
- `([^\n]+)` - counter-text until newline.

The forgiving parser

```
def _parse_counter_response(text):
    matches = _PIPE_RE.findall(text)
    if matches:
        # LAST match: if LLM wrapped its answer in preamble,
        # the last match is the real answer.
        hw_s, aw_s, tag, txt = matches[-1]
        hw = max(1, min(25, int(hw_s))) # clamp
        aw = max(1, min(25, int(aw_s)))
        tag = tag.capitalize()
        txt = txt.strip().strip('\'').strip('"')
        if "CONCEDE" in txt.upper():
            return {"human_weight":0,"llm_weight":0,
                    "llm_val":"Logic","llm_text":"CONCEDE",
                    "parsed":True}
        return {"human_weight":hw,"llm_weight":aw,
                "llm_val":tag,"llm_text":txt,"parsed":True}

    # Fallback: no pipe pattern. Use ONLY last line.
    # Prevents chain-of-thought from leaking.
    lines = [ln.strip() for ln in text.splitlines() if ln.strip()]
    fallback = lines[-1] if lines else text.strip()
    return {"human_weight":12,"llm_weight":12,
            "llm_val":"Logic","llm_text":fallback,
            "parsed":False}
```

10.4 The implicit-premise detector

Temperature: 0.4 here vs 0.7 for counters

Premise detection is analytic: same input -> same output. 0.4 is below the creative threshold (~0.7) but above fully deterministic. Counter generation is creative; 0.7 fits. Premise detection is extractive; 0.4 fits.

Output format

PREMISE_TEXT | CONFIDENCE | ONE_SENTENCE_JUSTIFICATION

If self-contained:

NONE | 0.0 | The argument states its grounds explicitly.

Confidence < 0.10 is also treated as NONE (defensive).

UI integration

If found=True, the UI shows the premise as a yellow callout with the confidence percentage and a checkbox 'attach this as a supporting node when I submit.' If checked, the engine gains a new node connected by a support edge. Implicit reasoning becomes visible in the formal graph - the Ku treatment.

Part 11 - Every View and Feature

11.1 The local debate view

File: views/local_debate.py, ~1019 lines, the most complex file. Renders the entire single-machine debate for HvAI and HvH-Local.

Major sections

- **Session-state init** - `_init_session_state()` sets every key so interactions never throw `KeyError`.
- **Top dashboard** - main-claim banner, mode indicator, A/B momentum percentages, inline End Debate button (5:1 column split).
- **Chat history** - iterates `st.session_state.messages`, renders each as a cloud-shaped bubble via `render_argument_bubble()`.
- **Logic map** - `render_logic_graph()` uses Graphviz to draw nodes and edges with status-coloured fills.
- **Composer** - multi-modal input (text, voice, image), value tag selector, target dropdown, Check Assumptions and Spell Check.
- **Move processing** - `_process_move` validates and adds nodes to the engine, then calls `evaluate_semantics`.
- **Battle history** - `_save_battle/_load_battle` persist debates to `unified_battle_history.json` (absolute path).
- **Undo** - `_undo_last_move` pops 2 in HvAI, 1 in HvH; rebuilds via `rebuild_from_messages`.
- **Post-debate analysis** - `_render_victory_screen` with trajectory chart, momentum chart, turning point, tag distribution, per-node table, TikZ export button.

Why so long?

Streamlit's reactive model puts every UI element in the same top-level function. Splitting requires managing session state across modules, which Streamlit makes awkward. Trade-off: one long file.

11.2 The AI vs AI view

File: views/ai_vs_ai.py, ~352 lines. Two LLM personas debate while the human watches and the engine adjudicates.

Major elements

- **Setup form** - pick topic, side A persona, side B persona, value tag for each side.
- **Run / Pause / Step / End** - human controls pacing.
- **`_run_one_turn()`** - one LLM reply for the active side, engine evaluates, active side flips.
- **Custom key** - `_get_client()` prefers `AIVAI_API_KEY` from secrets so AlvAI can use a separate Groq tier.

Why AlvAI exists

Pedagogically: watching two AIs debate your essay topic is a brainstorming aid. Methodologically: AlvAI runs are how `tools/run_self_play.py` stress-tests the engine on long chains.

11.3 The engine inspector (Defense Mode)

File: views/engine_inspector.py, ~393 lines. Five tabs:

- **Nodes** - table of every node: text, weight, tag, score, status.

- **Relations** - lists of attack and support edges.
- **MDP State** - current 3-tuple, chosen action, policy reasoning.
- **Diagnose** - pick a node, see per-edge contributions.
- **What If?** - sandbox to flip edges, change tags or weights, see hypothetical verdicts.

Why the inspector exists

Pedagogy: opens what the engine computes, makes it legible not magical. Debugging: when a verdict surprises, the inspector shows the cause. Defence: the inspector is a live demo of the symbolic-neural separation - you can show the engine has scores not coming from the LLM.

Neural/Symbolic legend

`_render_legend()` shows a red badge for Neural and a yellow badge for Symbolic. Every metric is tagged. Layer separation visible in real time.

11.4 Smaller features

Undo Last Move

`_undo_last_move` pops 2 messages in HvAI (human + AI pair) or 1 in HvH/AlvAI, then rebuilds the engine from scratch via `rebuild_from_messages`. Simpler than reversing the relaxation, and the engine is fast enough that it's invisible to the user.

Post-debate analysis

`_render_victory_screen` renders five panels: trajectory of main-claim score per turn, per-turn side momentum, turning-point move (the move that flipped the verdict), tag distribution per side, per-node table. Trajectory comes from `_compute_trajectory(messages)` which replays move-by-move via `rebuild_from_messages`.

TikZ export

`tikz_export.py`, ~183 lines. Emits LaTeX TikZ source for the argument graph: layered top-down BFS layout, red-to-yellow-to-green fills by score, solid red attacks, dashed blue supports. Output goes straight into your thesis's figures folder.

Spell check

`spellcheck.py`, ~95 lines, wraps `pyspellchecker` with a domain whitelist of project terms (`graphviz`, `llm`, `groq`, `llama`, `whisper`, `tikz`, `mdp`, `dung`, ...). Falls back to no-op if `pyspellchecker` is missing. Used for the graph visualisation only - the AI prompt already tells the LLM to ignore typos.

Check Assumptions (implicit-premise detector)

UI button that calls `detect_implicit_premise()`. Shows the premise, confidence, and an 'attach as supporter' checkbox. Operationalises the Ku 2025 diagnosis.

Part 12 - Evaluation, every number explained

12.1 The 60-scenario audit

60 hand-crafted scenarios across four topologies, 15 each:

- **LIN** - linear attack-only chains of varying depth.
- **STR** - multiple attackers converging on one root.
- **BIP** - interleaved attacks and supports.
- **LNG** - 10+ node debates mixing the above.

Each scenario has a hand-labelled verdict. `tools/run_evaluation.py` runs every scenario through the engine, thresholds, compares.

Headline numbers

Topology	Agreement
Linear chains	12/15 (80.0%)
Star attacks	14/15 (93.3%)
Bipolar mixed	12/15 (80.0%)
Long mixed	13/15 (86.7%)
Overall	51/60 (85.0%)

Why STR strongest, LIN/BIP weakest

STR is 'one claim, many attackers' - the formula is symmetric in attackers, handled cleanly. LIN has long chains where gradual propagation diverges from classical Dung. BIP adds supports that can land partially under attack and produce supporter residuals.

12.2 The ablation study

Same 60 scenarios with each component disabled, via `tools/run_ablation.py`.

Disabled component	Verdicts changed	Mean Delta s
Support relation R_{sup}	16/60 (26.7%)	0.206
Value-tag multipliers μ	2/60 (3.3%)	0.024
Per-argument weights w	5/60 (8.3%)	0.074
Gradual (use Dung grounded)	25/60 (41.7%)	0.525

Two findings to memorise:

- **Bipolarity is not decorative.** Disabling supports flips 26.7% of verdicts.
- **Your engine genuinely differs from classical Dung.** Replacing gradual with grounded flips 41.7% of verdicts.

Why μ only flips 3.3%

Most scenarios are uniform-tagged within a side, so μ ratios are 1. The multipliers only matter for cross-tag interactions (~10 of 60). Of those, only 2 are close enough to 0.5 to flip. Small ablation delta is a feature of the scenario set, not evidence that tags do not matter when learners use them.

12.3 Unit tests

101 passing assertions:

- **Engine** - bounds $[0,1]$, classical agreement, value-tag asymmetry direction, bipolar support, convergence, diagnose, dict round-trip, weight clamping.
- **MDP** - 18 states, 4 actions, policy covers all, LOSING $\rightarrow$ STEP_BACK, HIGH $\rightarrow$ VALUE_REFRAKE, transition sums = 1.
- **Scenario regression** - all 60 valid, agreement stays above 80%.
- **TikZ export** - empty engine returns a comment, all nodes appear, distinct edge styles, LaTeX escapes.

12.4 Discrepancy analysis - the 15% that disagree

Pattern 1: even-depth chain leniency (LIN-04, LIN-14)

Even-depth chains: classical Dung says root OUT, my engine says IN. Why: gradual propagates partial scores. An attacker hit by another attacker is not zero but $1/(1+\alpha)$, leaving a residual the chain's last node cannot push to zero. Even-depth is more lenient than classical.

Pattern 2: weight-asymmetry floor (LIN-10, BIP-08, BIP-12)

When a target is hit by an attacker several times heavier, no light counter can rescue it. Denominator $1+\alpha/w$ grows large with attacker weight; lightweight defender adds only a small sigma. Score floor below 0.5.

Pattern 3: bipolar supporter residual (BIP-10, LNG-09, LNG-13, STR-13)

When a supporter of the main claim is itself attacked, classical intuition says it is neutralised. Gradual keeps a residual support alive because the attacked supporter's score is reduced not extinguished.

Practice answer: the three patterns are not bugs. They are the empirical signature of what gradual bipolar adds to classical Dung. The 15% disagreement is the price of having a continuous evaluator that gives learners gradient feedback rather than a binary one that says only IN or OUT.

Part 13 - Threats to Validity and Future Work

13.1 Threats to validity

Threat 1: single-rater ground truth

All 60 labels are mine. No inter-rater agreement. If my labels are biased toward what my engine outputs, the 85% is inflated. `tools/make_rater_template.xlsx` exists for multi-rater collection with Cohen's kappa, but I have not collected the data.

Threat 2: no user study

I built the tutor; I have not measured whether it teaches better than a baseline. Learning gain, transfer, satisfaction all unmeasured.

Threat 3: hand-tuned parameters

Value multipliers (1.2/1.1/1.0/0.8), score thresholds (0.5, 0.65, 0.40), fatigue multipliers (0.85/0.65), and MDP transitions are all hand-chosen. None learned from data.

Threat 4: scenario coverage

60 scenarios is small. Four topologies chosen by me may not represent real debates. Adversarial scenarios designed to break the engine were not in the test set.

Threat 5: LLM dependence

All AI behaviour depends on LLaMA-3 70B / 90B Vision via Groq at my chosen temperatures. Different LLMs may behave differently. No cross-LLM generalisation tested.

Threat 6: no latency benchmark

Groq latency varies. I have not run timed benchmarks. Classroom ITS needs sub-2-second response under load; not measured.

13.2 Future work

Future 1: multi-rater scenario evaluation

3-5 independent expert raters. Cohen's kappa per topology and overall. Publish IRR alongside engine-vs-human agreement.

Future 2: controlled user study

Pre/post-test design comparing tutor versus baseline LLM-chat. Outcomes: argument quality on transfer task, confidence, time-on-task.

Future 3: empirical re-estimation of P

Replace hand-coded transition with learned-from-logs. The stub function `update_transition_model_from_logs` already exists.

Future 4: per-classroom value-tag tuning

Teachers re-weight multipliers per classroom - Ethics highest for moral-reasoning, Emotion highest for creative-writing. Bench-Capon's audience-relativity as a deployable feature.

Future 5: convergence proof

Formal Banach proof of fixed-point existence and uniqueness for the bipolar gradual formula. The sketch is in Chapter 6; a full proof with explicit bounds would be a stronger theoretical contribution.

Future 6: better LLM grounding

Current grounding relies on pipe-format emission and Rules A-D being respected. Constrained-decoding or structured-output API would remove the need for the forgiving parser.

Part 14 - Glossary

Alphabetical. If you can give a one-sentence definition of every entry here, you are ready.

A - D

- **Abstract Argumentation Framework (AAF)** - Dung 1995 pair $\langle A, R \rangle$ of arguments and attack relation.
- **Acceptable argument** - one whose every attacker is itself attacked by something in the set.
- **Action (MDP)** - one of the four hint strategies.
- **Ablation study** - re-running with one component removed.
- **Admissible set** - conflict-free set in which every member is acceptable.
- **alpha(a)** - attack contribution sum.
- **Audience preference** - Bench-Capon ordering on values.
- **BKT** - Bayesian Knowledge Tracing; probabilistic skill tracking in ITS.
- **Bellman optimality equation** - recursive characterisation of the optimal value function.
- **Bipolar AF** - Cayrol-Lagasque-Schiex extension with a support relation.
- **CONCEDE sentinel** - exact string '0|0|Logic|CONCEDE'; the LLM's only honest exit.
- **Conflict-free set** - no internal attacks.
- **Defeasible reasoning** - conclusions can be retracted; synonym non-monotonic.
- **derive_state()** - engine $\rightarrow$ MDP 3-tuple state.
- **Discount (gamma)** - in $[0,1)$, how much future rewards matter.
- **Dung's framework** - $F = \langle A, R \rangle$ abstract argumentation.

E - N

- **Extension (Dung)** - a set of jointly acceptable arguments.
- **Fixed-point iteration** - iterative algorithm for $x = f(x)$.
- **Gradual semantics** - continuous scores in $[0,1]$ instead of binary IN/OUT.
- **Grounded extension** - smallest complete extension; always unique; most sceptical.
- **h-categorizer** - Besnard-Hunter's $s(a) = 1/(1+\text{sum atk})$.
- **Hint Factory** - Stamper et al.'s data-driven hint technique.
- **Implicit premise** - unstated assumption an argument relies on.
- **IN / OUT** - binary statuses from thresholding scores at 0.5.
- **Logic Advocate Tutor** - the name of your project.
- **Main Claim (Msg_1)** - proponent's central thesis; its verdict decides the debate.
- **MDP** - Markov Decision Process; sequential decision making.
- **Momentum bar** - cumulative weight of each side's surviving arguments.
- **mu** - value-tag multiplier function.
- **Non-monotonic logic** - logic where conclusions can be retracted.

O - Z

- **On-demand hint** - appears only on request; Stamper principle.
- **Pedagogical guarantee** - the LLM cannot lie about the verdict because the engine decides.
- **Pipe protocol** - HumanScore|AIScore|ValueTag|CounterText.
- **Policy (MDP)** - states $\rightarrow$ actions mapping.
- **Post-debate analysis** - trajectory/momentum/turning-point screen.
- **Preference-Based AF** - Amgoud-Cayrol weighted extension.
- **R_att, R_sup** - attack and support relations.
- **Rhetorical rigidity** - Ku 2025 term for LLM refusing to concede.
- **sigma(a)** - support contribution sum.
- **State (MDP)** - 3-tuple (own_status, opponent_pressure, hint_streak).
- **Streamlit session_state** - dict persisting across Streamlit re-runs in one tab.
- **Symbolic vs Neural** - symbolic = deterministic Python; neural = LLM-generated text.
- **Synchronous relaxation** - fixed-point iteration updating from the PREVIOUS iteration's vector.
- **Toulmin model** - 1958 six-component argument structure.
- **Value-Based AF** - Bench-Capon 2003 framework with value tags.
- **Value iteration** - exact MDP solver via Bellman updates.
- **VALUE_WEIGHTS dict** - {Fact:1.2, Logic:1.1, Ethics:1.0, Emotion:0.8}.
- **w(a)** - integer weight in $\{1, \dots, 25\}$ per argument.

Part 15 - Mock Viva Script

A simulated 25-minute defence. Read the supervisor's question, answer aloud BEFORE looking. Repeat twice.

Opening - the 'sell me on your project' minute

Q. Tell me what you built.

A. I built a Streamlit web application called the Logic Advocate Tutor. It is a debate environment where a student can debate another human or an LLM. Every move is judged by a deterministic bipolar gradual argumentation engine I wrote in pure Python, and every hint is chosen by a small Markov Decision Process. The LLM writes the prose but never decides who is winning. That separation is the project's central pedagogical claim.

Q. Why did you build it?

A. Two motivations. Conversational AI is unreliable at admitting defeat - Ku et al. 2025 measured this and called it rhetorical rigidity. A student debating a chatbot cannot trust the chatbot's verdict. And computational argumentation theory is 30 years old but has very few deployed tutors. This project bridges both gaps by using the symbolic engine to police the LLM's claims.

Theoretical foundations

Q. Whose framework is your engine based on?

A. Dung's 1995 abstract argumentation framework is the irreducible core. On top I added Cayrol-Lagasque-Schiex's bipolar (supports), Amgoud-Cayrol's preference-based weights, Bench-Capon's value-tagged framework, and Cayrol's gradual semantics. With extras off, the engine reduces to Besnard-Hunter's 2001 h-categorizer.

Q. What does Dung's framework say?

A. $F = \langle A, R \rangle$: a finite set of abstract arguments and the attack relation. The grounded extension iteratively accepts arguments whose every attacker is attacked by something already accepted. My 0.5 threshold approximates grounded when the extras are off.

Q. Why gradual instead of classical Dung?

A. Pedagogically: 'your argument is OUT' is uninformative; students cannot distinguish narrowly defeated from thoroughly demolished. Continuous scores give gradient feedback. Empirically: my ablation showed 41.7% of verdicts change between gradual and grounded, so the choice matters.

The engine

Q. Write your formula.

A. $s(a) = \min(1, (1 + \sigma(a)/w(a)) / (1 + \alpha(a)/w(a)))$. Sigma is support sum, alpha is attack sum. Both include the $\mu(v(b))/\mu(v(a))$ ratio. The cap keeps the score in $[0, 1]$.

Q. Why is it a fixed point?

A. Every score depends on other scores. The iteration starts at 1, applies the formula synchronously - every score this iteration reads from the previous iteration's vector - until the vector stops changing. Synchronous makes the result independent of node insertion order. Converges in 2-3 iterations on all 60 scenarios.

Q. Why $\min(1, \dots)$?

A. Three reasons. Cap preserves $[0, 1]$, keeps the UI gradient defined. It generalises the h-categorizer, which is already in $[0, 1]$, so the classical limit holds. And it makes the formula monotone: more support never lowers the score.

The MDP

Q. Why an MDP?

A. Hint selection is inherently sequential decision-making. State is the learner's strategic situation, actions are hint strategies, reward is whether the next move improves the engine's verdict. Value iteration is exact for the small discrete state space, so I get the optimal policy in microseconds.

Q. Why 18 states?

A. Three buckets with intentionally small cardinality. `own_status` (3) ties to the reward. `opponent_pressure` (2) separates comfortable wins from precarious ones. `hint_streak` (3) implements Stamper's non-evaluative principle. 18 is the minimum strategic context; smaller loses signal, larger would not generalise from the small data I have.

Q. How is the reward computed?

A. $R = +1$ if next `own_status` WINNING, 0 if TIED, -1 if LOSING, minus 0.05 hint cost. `own_status` comes from `engine.statuses[Msg_1]` via `derive_state`. So the reward is the engine's verdict.

The AI layer

Q. How do you stop the LLM from lying about the verdict?

A. I don't ask. The engine decides; the LLM is never asked the question. The LLM rates persuasiveness on 1-25; those go IN as weights, not OUT as verdicts. After every move, `evaluate_semantics` runs. The UI reads the engine. The LLM is structurally unable to lie about a question it is not asked.

Q. What stops the guarantee from being broken in a later change?

A. The boundary lives in `_build_transcript()` in `ai_agent.py`. That function reads messages and never engine state. If I added engine state to the prompt, the guarantee breaks. It would be a deliberate code change, not accidental drift. Unit tests do not currently enforce this - something to add.

Q. Why the pipe protocol?

A. Need structured output from the LLM. Pipe is simple to parse with a regex, and the regex takes the LAST match so preamble does not break it. Fallback uses only the last line so rambling does not leak into the chat bubble.

Evaluation

Q. Headline result?

A. 85% agreement between my engine and my hand-labelled ground truth on 60 scenarios across four topologies. The 15% disagreement is three documented patterns, all properties of gradual semantics not bugs.

Q. Is 85% good?

A. It is a baseline. No published comparable benchmark for bipolar gradual evaluators. The number is useful relative to ablation: turning off supports or graduality drops it. Stronger evaluation needs multiple raters and a kappa statistic - the spreadsheet is built but data not collected.

Q. What about user studies?

A. Not yet done. The thesis claims a pedagogical guarantee, not a measured learning gain. A controlled study is future work. The system is defensible on theoretical and engineering grounds; the empirical claim is about engine behaviour not student learning.

Limitations

Q. What would you change if starting over?

A. Log student data from day one to enable MDP P re-estimation. And constrained-decoding for the LLM to enforce the pipe format at the API level rather than relying on a forgiving parser.

Q. Biggest weakness?

A. Single-rater ground truth. All 60 labels are mine. 85% agreement is the engine agreeing with its author, the weakest evaluation form. The rater spreadsheet computes Cohen's kappa automatically when multi-rater data arrives; I have not collected it.

Closing

Q. One thing the project taught you?

A. The boundary between conversational AI and formal logic is best policed by code, not by prose. Asking a language model to be honest about whether it is losing is asking the wrong agent the wrong question. Making it unable to lie, by depriving it of the privilege of computing the verdict, turns out to be much easier and much more reliable.

Part 16 - Question Bank by Topic

100+ practice questions. Cover one topic per session. Read the question, look away, answer aloud, compare. Move on.

Topic A - Foundations and literature

Q. State Dung's framework formally.

A. $F = \langle A, R \rangle$ where A is a finite set of arguments and R is the attack relation on A .

Q. Conflict-free set?

A. Subset of A with no internal attacks.

Q. Grounded extension?

A. Smallest complete extension; always unique; the most sceptical Dung verdict.

Q. Preferred vs stable?

A. Preferred is maximal complete; stable additionally attacks every non-member. Stable may not exist; preferred always does.

Q. Why is monotonic logic insufficient for argumentation?

A. Because real arguments are defeasible: new evidence retracts old conclusions. Monotonic logic cannot retract.

Q. What did Toulmin add?

A. Explicit modelling of WARRANT and REBUTTAL as separate components.

Q. Why not use Toulmin directly?

A. Mapping free text to all six components is an unsolved NLP problem. Dung's abstraction is computationally tractable.

Q. What did Besnard-Hunter add?

A. A continuous numerical evaluator (h-categorizer) with scores in $(0,1]$.

Q. h-categorizer formula?

A. $h(a) = 1 / (1 + \sum h(b) \text{ for every } b \text{ that attacks } a)$.

Q. Score of an isolated argument under h?

A. 1, because the sum over attackers is empty.

Q. What did Bench-Capon add?

A. Value tags and audience preferences: arguments promote different values; attacks succeed only if the audience does not prefer the target's value to the attacker's.

Q. What did Amgoud-Cayrol add?

A. Per-argument preferences so stronger arguments resist defeat from weaker.

Q. What did Cayrol-Lagasquie-Schiex add?

A. The bipolar extension: a support relation alongside attack.

Q. What did Cayrol's graduality paper add?

A. Real-valued strengths replacing categorical IN/OUT.

Q. Who built the Hint Factory?

A. Stamper, Eagle, Barnes, Croy in 2011.

Q. Two Hint Factory principles you adopted?

A. On-demand (hints on request) and non-evaluative (suggest direction, not 'you are wrong').

Q. What did Ku 2025 find?

A. LLMs assigned debate stances exhibit rhetorical rigidity and miss implicit premises routinely.

Q. How does your work go beyond Ku?

A. Ku diagnoses; you treat. Verdict taken out of LLM's hands entirely; Check Assumptions UI surfaces implicit premises into the formal graph.

Topic B - The engine

Q. Write the formula.

A. $s(a) = \min(1, (1 + \sigma(a)/w(a)) / (1 + \alpha(a)/w(a)))$.

Q. What is alpha?

A. Sum over attackers b of $s(b)*w(b)*\mu(v(b))/\mu(v(a))$.

Q. What is sigma?

A. Sum over supporters b of $s(b)*w(b)*\mu(v(b))/\mu(v(a))$.

Q. Value-tag multipliers?

A. Fact 1.2, Logic 1.1, Ethics 1.0, Emotion 0.8.

Q. Threshold for IN/OUT?

A. 0.5: scores at or above are IN, below are OUT.

Q. Why cap at 1?

A. Preserve $[0, 1]$ interpretation, keep UI gradient defined.

Q. Why init at 1, not 0?

A. The relaxation only pushes scores down as attacks accumulate. Starting at 1 has the right asymptotic behaviour from iteration 1.

Q. Why synchronous, not asynchronous?

A. Synchronous makes the result independent of node insertion order, important for reproducibility.

Q. How fast does it converge?

A. 2-3 iterations on the 60 scenarios. 10 is a safety ceiling with early-termination at $\delta < 1e-6$.

Q. Empty engine?

A. Loops over nothing. Returns gracefully with empty dicts.

Q. Self-attacker?

A. Iterates to $x = 1/(1+x)$, giving $(\sqrt{5}-1)/2 = 0.618$.

Q. Support cycle?

A. Would grow above 1 except for the cap; saturates at 1.

Q. Why alpha denominator and sigma numerator?

A. Design choice. Attacks suppress (push toward 0), supports boost (push toward 1). Asymmetric placement captures the intuition.

Q. Classical limit?

A. Empty support, $w=1$, $\mu=1$: formula reduces to h-categorizer.

Q. Invariant on $s(a)$?

A. $0 < s(a) \leq 1$ at every iteration.

Topic C - The MDP

Q. What kind of system is your hint mechanism?

A. Markov Decision Process; 18 states, 4 actions, hand-coded P, engine-derived R, discount 0.9.

Q. Three state buckets?

A. `own_status` (W/T/L), `opponent_pressure` (LOW/HIGH), `hint_streak` (ZERO/ONE/TWO_PLUS).

Q. Four actions?

A. `STRATEGIC_ATTACK`, `DEFENSIVE_REINFORCE`, `VALUE_REFRAME`, `STEP_BACK`.

Q. Reward formula?

A. $R = (+1 \text{ W}, 0 \text{ T}, -1 \text{ L}) - 0.05 \text{ hint cost}$.

Q. Where does `own_status` come from?

A. `engine.statuses[Msg_1]` via `derive_state`. Polarity flipped for Side B.

Q. Why $\gamma = 0.9$?

A. Standard textbook value. Future rewards matter but less than immediate. Must be < 1 for value iteration to converge.

Q. How is the policy stored?

A. Dict mapping state to action. Cached at module import via `solve()`. $O(1)$ lookup at runtime.

Q. Why expert-informed P?

A. No logged student data yet. `update_transition_model_from_logs` is a stub for future empirical re-estimation.

Q. Why value iteration not Q-learning?

A. Exact for small discrete spaces. 18 states converge in microseconds.

Q. What does the fatigue multiplier do?

A. Reduces hint effectiveness with streak ($0.85 \times \text{ONE}$, $0.65 \times \text{TWO_PLUS}$). Implements Stamper non-evaluative.

Q. Which action when LOSING under HIGH pressure?

A. `VALUE_REFRAME` or `STEP_BACK` depending on streak.

Topic D - The AI layer

Q. Rules A-D?

A. A: accept personal experience. B: score honestly. C: concede when no real counter. D: ignore typos and casual phrasing.

Q. Pipe protocol?

A. `HumanScore|AIScore|ValueTag|CounterText`. Exactly one line, no preamble.

Q. CONCEDE sentinel?

A. Exact string '0|0|Logic|CONCEDE'. The LLM's only honest exit.

Q. How does the parser tolerate preamble?

A. Takes the LAST match of the pipe regex. Preamble before the final pipe line does not break it.

Q. Fallback behaviour?

A. If no pipe match, use ONLY the last line of the response, never the whole text. Prevents chain-of-thought leak.

Q. Counter temperature?

A. 0.7 - creative writing range.

Q. Premise-detection temperature?

A. 0.4 - analytic / extractive range.

Q. Premise detector output format?

A. PREMISE_TEXT | CONFIDENCE | JUSTIFICATION. Or NONE | 0.0 | reason if self-contained.

Q. How is a detected premise added to the engine?

A. Learner checks 'attach as supporter'; on submit, a new node is added with a support edge to the original argument.

Q. Where is the symbolic-neural boundary?

A. In `_build_transcript()`: reads messages but never engine state.

Topic E - Architecture and views

Q. Four layers?

A. Presentation, Dialogue, Logic (symbolic), AI (neural).

Q. What does each view do?

A. `local_debate.py`: HvAI and HvH-local. `ai_vs_ai.py`: AlvAI spectator. `engine_inspector.py`: Defense Mode with 5 tabs.

Q. How does Undo work?

A. Pops 2 messages in HvAI (human + AI pair) or 1 in HvH/AlvAI. Rebuilds the engine via `rebuild_from_messages`. Simpler than reversing the relaxation.

Q. Post-debate analysis panels?

A. Trajectory of main-claim score; per-turn momentum; turning-point move; tag distribution per side; per-node table; TikZ export.

Q. How is the trajectory computed?

A. `_compute_trajectory` replays move-by-move via `rebuild_from_messages`, recording per-turn snapshots.

Q. Five Inspector tabs?

A. Nodes, Relations, MDP State, Diagnose, What If?

Q. What If? sandbox?

A. Flip edge type, change tag, change weight; see hypothetical verdict without modifying the live engine.

Q. TikZ exporter?

A. Emits LaTeX TikZ source: layered BFS layout, red-to-green fills by score, solid red attacks, dashed blue supports.

Q. Spellcheck?

A. pyspellchecker wrapper with a project-term whitelist. Falls back to no-op if pyspellchecker missing.

Topic F - Evaluation

Q. Headline agreement?

A. 85% (51/60).

Q. Best topology?

A. Star attacks 93.3%.

Q. Worst topology?

A. Linear chains and bipolar mixed 80% each.

Q. Why STR easiest?

A. Formula is symmetric in attackers, handles many-to-one cleanly. Linear chains have long propagation paths where gradual diverges.

Q. Turning off supports?

A. Flips 26.7% of verdicts; mean $|\Delta s| = 0.206$.

Q. Replacing gradual with classical Dung?

A. Flips 41.7% of verdicts; mean $|\Delta s| = 0.525$.

Q. Turning off weights?

A. Flips 8.3%; mean $|\Delta s| = 0.074$.

Q. Turning off value multipliers?

A. Flips 3.3%; small because most scenarios are uniform-tagged within a side, so ratios are 1.

Q. Unit-test count?

A. 101 passing assertions.

Q. Three discrepancy patterns?

A. 1) Even-depth chain leniency. 2) Weight-asymmetry floor. 3) Bipolar supporter residual.

Q. Is the 15% disagreement a bug?

A. No. Each pattern is documented and traceable to a formula property. It is the empirical signature of what gradual bipolar adds to classical Dung.

Topic G - Limitations and meta

Q. Biggest threat to validity?

A. Single-rater ground truth.

Q. User-study evidence?

A. None yet. Future work.

Q. Learned parameters?

A. None. All hand-tuned: multipliers, thresholds, fatigue, transitions.

Q. What to do differently if starting over?

A. Log student data from day one. Use constrained-decoding for the pipe format.

Q. One sentence to remember?

A. The boundary between conversational AI and formal logic is best policed by code, not by prose.

End of Deep Study Guide. Good luck.